

Modelling and Explaining Electricity Prices in Europe

Analysis of France & Germany Spot Markets (QRT Challenge)

Sammy RABHINE

2026-05-22

Contents

Introduction	1
Data Processing	1
Missing Value Analysis	2
Variable Distribution	4
Exploratory Data Analysis (EDA)	7
Feature Engineering	24
Feature Selection	26
XGBOOST Modelling	27

Introduction

How can we explain the daily variations in electricity futures prices in France and Germany using meteorological, energy, and cross-border exchange factors?

Data Processing

Displaying the first 5 rows of the dataset

```
# Data loading
X_train <- read_csv("X_train.csv")
y_train <- read_csv("y_train.csv")
X_test  <- read_csv("X_test_final.csv")

### Data Overview and Structure
# Display first 5 rows
X_train %>%
  head(5) %>%
  kable(caption = "Overview of the first 5 rows of the training dataset") %>%
  kable_styling(bootstrap_options = c("striped", "hover", "condensed"), full_width = F) %>%
  scroll_box(width = "100%")
```

ID	DAY_ID	COUNTRY	DE_CONSUMPTION	FR_CONSUMPTION	DE_FR_EXCHANGE	FR_DE_EXCHANGE
1054	206	FR	0.2100987	-0.4274582	-0.6065235	
2049	501	FR	-0.0223993	-1.0034525	-0.0220632	
1924	687	FR	1.3950351	1.9786654	1.0213052	
297	720	DE	-0.9833239	-0.8491980	-0.8395859	
1101	818	FR	0.1438074	-0.6170377	-0.9249900	

```
# Variable type verification
type_info <- data.frame(
  Variable = names(X_train),
  Type = sapply(X_train, class)
)

type_info %>%
  count(Type) %>%
  kable(caption = "Distribution of variable types") %>%
  kable_styling(full_width = F, position = "left")
```

Table 2: Distribution of variable types

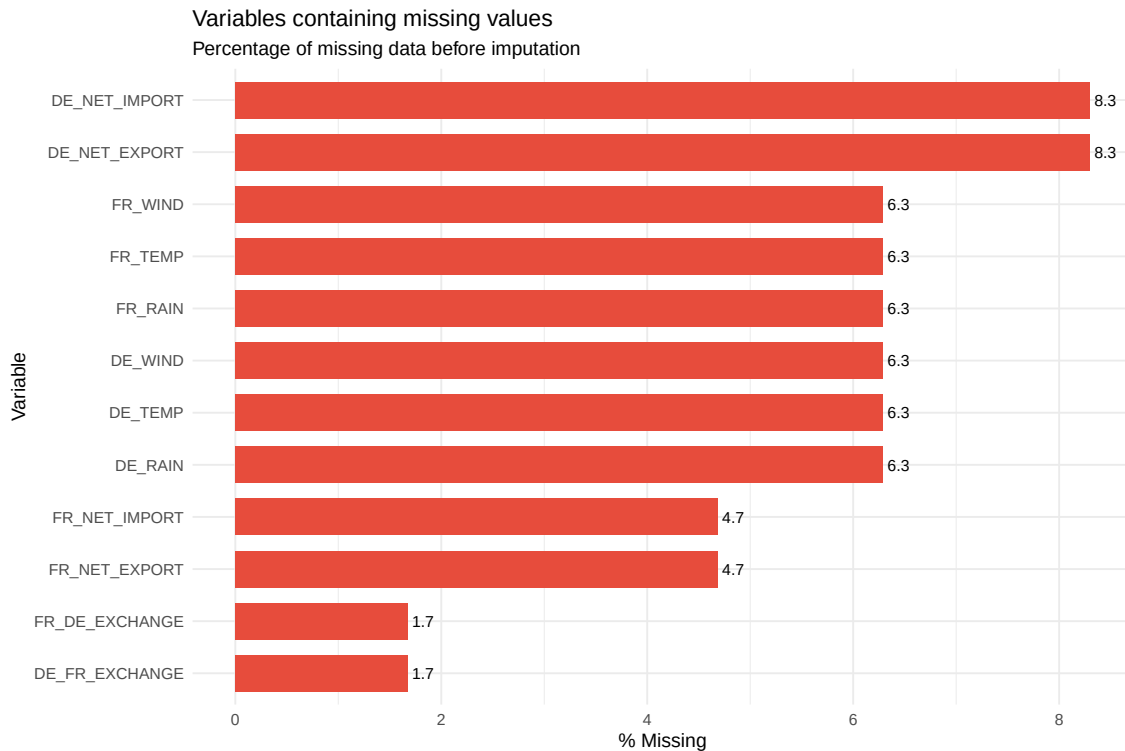
Type	n
character	1
numeric	34

I then wanted to see which variables had missing values.

Missing Value Analysis

```
### Missing Value Analysis
# X_train
na_summary <- X_train %>%
  summarise(across(everything(), ~sum(is.na(.)))) %>%
  pivot_longer(everything(), names_to = "Variable", values_to = "Nb_NA") %>%
  mutate(Percentage = (Nb_NA / nrow(X_train)) * 100) %>%
  filter(Nb_NA > 0) %>%
  arrange(desc(Percentage))

# Missing data visualisation
ggplot(na_summary, aes(x = reorder(Variable, Percentage), y = Percentage)) +
  geom_bar(stat = "identity", fill = "#e74c3c", width = 0.7) +
  coord_flip() +
  theme_minimal() +
  labs(
    title = "Variables containing missing values",
    subtitle = "Percentage of missing data before imputation",
    x = "Variable",
    y = "% Missing"
  ) +
  geom_text(aes(label = round(Percentage, 1)), hjust = -0.2, size = 3)
```



I then decided to proceed with imputation of the missing values.

```
# 1. Variable group definitions
meteo_vars <- c("FR_WINDPOW", "DE_WINDPOW", "FR_TEMP", "DE_TEMP", "FR_RAIN", "DE_RAIN")

exchange_vars <- c("FR_DE_EXCHANGE", "DE_FR_EXCHANGE")

net_vars <- c("FR_NET_IMPORT", "FR_NET_EXPORT", "DE_NET_IMPORT", "DE_NET_EXPORT")

# 2. WEATHER imputation (Linear interpolation)
X_train <- X_train %>%
  mutate(across(
    any_of(meteo_vars),
    ~ na.approx(.x, na.rm = FALSE)
  ))

# 3. EXCHANGE imputation (Linear interpolation)
X_train <- X_train %>%
  mutate(across(
    any_of(exchange_vars),
    ~ na.approx(.x, na.rm = FALSE)
  ))

# 4. NET FLOWS imputation (Interpolation with cap of 3 + Rolling Mean)
X_train <- X_train %>%
  # Step A: Interpolation for small gaps (maxgap = 3)
  mutate(across(
    any_of(net_vars),
    ~ na.approx(.x, na.rm = FALSE, maxgap = 3)
  )) %>%
```

```

# Step B: Rolling mean for remaining gaps
mutate(across(
  any_of(net_vars),
  ~ ifelse(
    is.na(.x),
    rollapply(.x, width = 7, FUN = mean, fill = NA, align = "center", partial = TRUE),
    .x
  )
))

# 5. Edge cleaning (Last Observation Carried Forward / Backward)
vars_to_clean <- c(meteo_vars, exchange_vars, net_vars)

X_train <- X_train %>%
  mutate(across(
    any_of(vars_to_clean),
    ~ na.locf(.x, na.rm = FALSE)
  )) %>%
  mutate(across(
    any_of(vars_to_clean),
    ~ na.locf(.x, fromLast = TRUE, na.rm = FALSE)
  ))

# Final check
print("Remaining missing values:")

## [1] "Remaining missing values:"
print(colSums(is.na(X_train %>% select(any_of(vars_to_clean)))))

##      FR_WINDPOW      DE_WINDPOW      FR_TEMP      DE_TEMP      FR_RAIN
##           0           0           0           0           0
##      DE_RAIN FR_DE_EXCHANGE DE_FR_EXCHANGE FR_NET_IMPORT FR_NET_EXPORT
##           0           0           0           0           0
## DE_NET_IMPORT DE_NET_EXPORT
##           0           0

# Merge X_train and y_train
df_train <- X_train %>%
  left_join(y_train, by = "ID")

```

After performing the variable imputation, I wanted to see how the variables were distributed (features + target).

Variable Distribution

```

### Feature Distribution Matrix
# Selecting numerical variables (excluding ID and Target) for visualisation
features_long <- df_train %>%
  select(-ID, -TARGET) %>%
  select(where(is.numeric)) %>%
  pivot_longer(cols = everything(), names_to = "Variable", values_to = "Value")

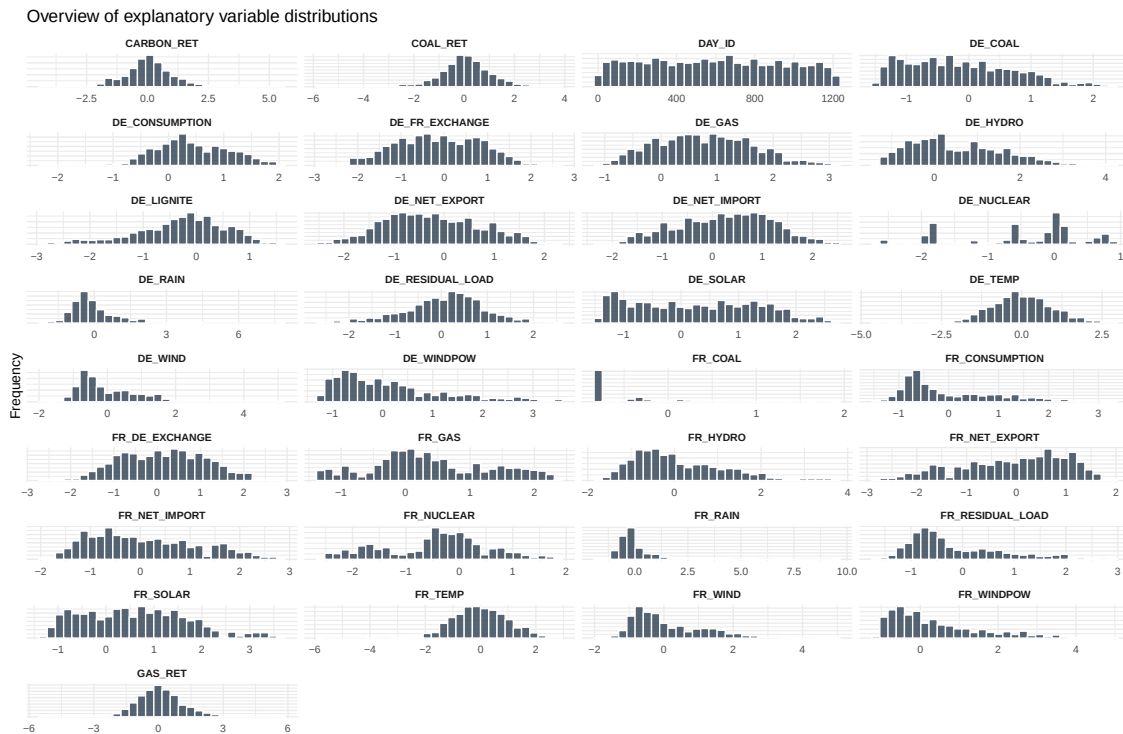
# Global visualisation
ggplot(features_long, aes(x = Value)) +

```

```

geom_histogram(bins = 30, fill = "#2c3e50", color = "white", alpha = 0.8) +
facet_wrap(~Variable, scales = "free", ncol = 4) +
theme_minimal(base_size = 8) +
theme(
  strip.text = element_text(face = "bold"),
  axis.text.y = element_blank(),
  axis.ticks.y = element_blank()
) +
labs(
  title = "Overview of explanatory variable distributions",
  x = "", y = "Frequency"
)

```



The distributions of the explanatory variables show heterogeneous behaviour: market and price variables display centred and nearly symmetrical distributions, while meteorological and renewable energy production variables show asymmetric distributions with pronounced tails, reflecting the non-linearity and intermittency of the electricity system.

```

cols_pays <- c("FR" = "#3498db", "DE" = "#e74c3c")

### Distribution of the Target Variable (TARGET)
stats_target <- df_train %>%
  group_by(COUNTRY) %>%
  summarise(
    Mean = mean(TARGET),
    Median = median(TARGET),
    StdDev = sd(TARGET),
    Min = min(TARGET),
    Max = max(TARGET)
  )

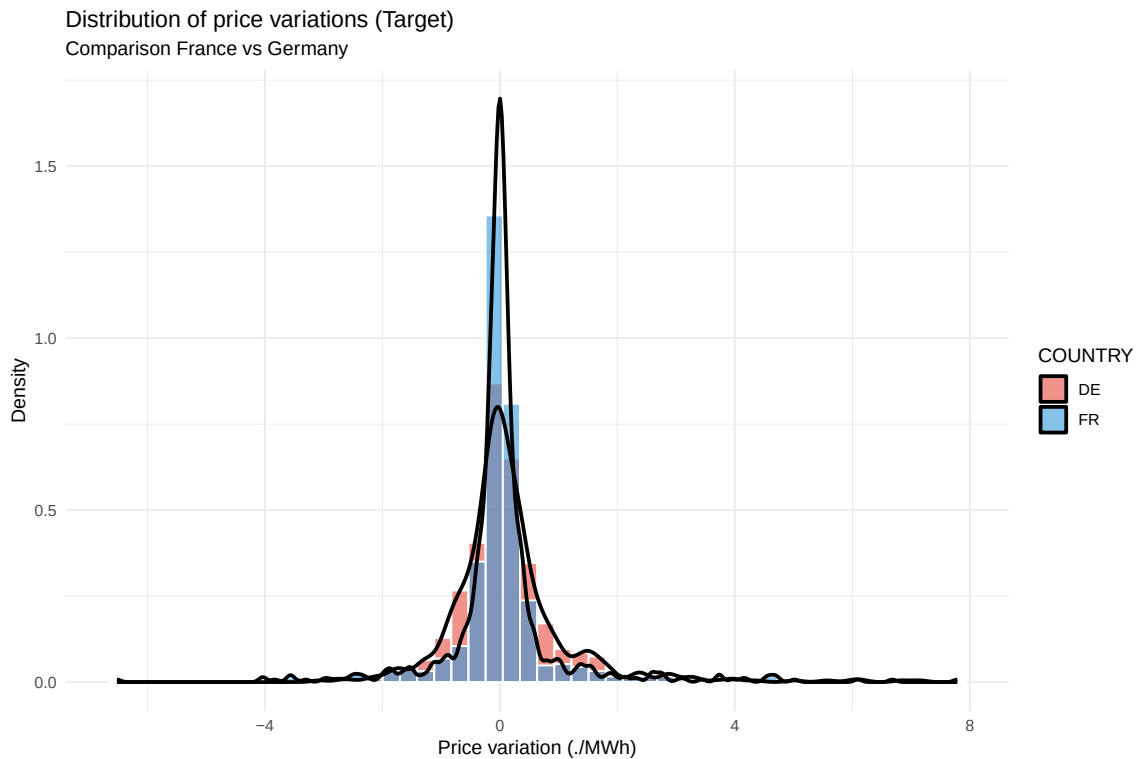
```

```
print(kable(stats_target, digits = 2, caption = "Descriptive statistics of the Target"))
```

```
##
##
## Table: Descriptive statistics of the Target
##
## | COUNTRY | Mean | Median | StdDev | Min | Max |
## | :----- | :--- | :----- | :----- | :--- | :--- |
## | DE      | 0.15 | 0.01 | 1.05 | -3.08 | 7.14 |
## | FR      | 0.05 | 0.00 | 1.02 | -6.52 | 7.79 |
```

```
# Histogram + Density by Country
```

```
ggplot(df_train, aes(x = TARGET, fill = COUNTRY)) +
  geom_histogram(aes(y = ..density..), bins = 50, alpha = 0.6, position = "identity", color = "white") +
  geom_density(alpha = 0, size = 1) +
  scale_fill_manual(values = cols_pays) +
  scale_color_manual(values = cols_pays) +
  theme_minimal() +
  labs(
    title = "Distribution of price variations (Target)",
    subtitle = "Comparison France vs Germany",
    x = "Price variation (€/MWh)",
    y = "Density"
  )
```



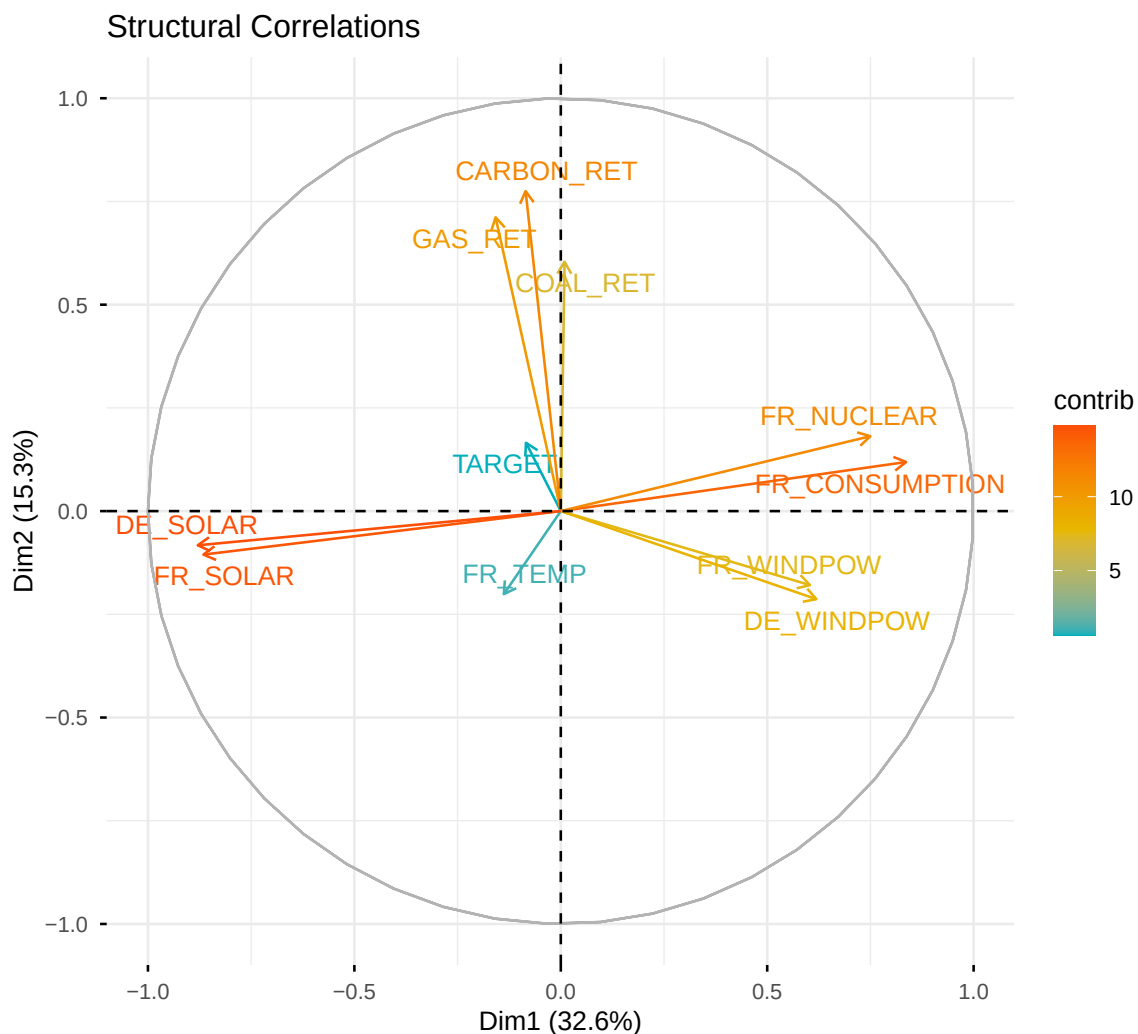
The price distributions reveal strong heterogeneity and increased volatility in Germany, suggesting distinct market dynamics. This observation, combined with the structural differences in the French and German electricity mixes, motivates a comparative exploratory analysis aimed at highlighting the main price formation mechanisms.

Exploratory Data Analysis (EDA)

```
# Selection of physical and economic variables
data_pca <- df_train %>%
  select(TARGET, FR_CONSUMPTION, FR_WINDPOW, FR_SOLAR, FR_TEMP,
         FR_NUCLEAR, GAS_RET, COAL_RET, CARBON_RET,
         DE_WINDPOW, DE_SOLAR)

res.pca <- PCA(data_pca, scale.unit = TRUE, graph = FALSE)

fviz_pca_var(res.pca,
  col.var = "contrib",
  gradient.cols = c("#00AFBB", "#E7B800", "#FC4E07"),
  repel = TRUE,
  title = "Structural Correlations")
```



The correlation circle highlights a clear structure of the electricity system: the first dimension contrasts renewable energies (wind, solar) against demand and dispatchable production variables, while the second dimension is dominated by fossil fuel prices, indicating that price formation results from the interaction between physical constraints and market signals.

```

df_regime <- df_train %>%
  filter(COUNTRY %in% c("FR", "DE")) %>%
  mutate(
    RENEWABLE = ifelse(COUNTRY == "FR",
                      FR_WINDPOW + FR_SOLAR,
                      DE_WINDPOW + DE_SOLAR),

    CONSUMPTION = ifelse(COUNTRY == "FR",
                        FR_CONSUMPTION,
                        DE_CONSUMPTION),

    RESIDUAL_LOAD = ifelse(COUNTRY == "FR",
                          FR_RESIDUAL_LOAD,
                          DE_RESIDUAL_LOAD),

    RENEW_RATIO = RENEWABLE / (CONSUMPTION + 1e-6),

    GAS = GAS_RET,
    COAL = COAL_RET,

    NET_FLOW = ifelse(COUNTRY == "FR",
                     FR_NET_IMPORT,
                     -DE_NET_EXPORT)
  ) %>%
  select(
    COUNTRY,
    TARGET,
    RENEW_RATIO,
    RESIDUAL_LOAD,
    GAS,
    COAL,
    NET_FLOW
  ) %>%
  drop_na()

# Standardisation
X_scaled <- df_regime %>%
  select(-COUNTRY, -TARGET) %>%
  scale()

set.seed(123)

# 3 regimes
k <- 3
km <- kmeans(X_scaled, centers = k, nstart = 50)

df_regime <- df_regime %>%
  mutate(
    Cluster = factor(km$cluster,
                    labels = c("RES Regime", "Stress Regime", "Intermediate Regime"))
  )

# Economic profile of clusters
cluster_summary <- df_regime %>%

```



```

group_by(Cluster) %>%
  summarise(
    Median_Price = median(TARGET),
    RES_Surplus = median(RENEW_RATIO),
    Load = median(RESIDUAL_LOAD),
    Gas = median(GAS),
    .groups = "drop"
  )

cluster_summary

## # A tibble: 3 x 5
##   Cluster      Median_Price RES_Surplus   Load   Gas
##   <fct>          <dbl>         <dbl> <dbl> <dbl>
## 1 RES Regime      -0.0250        -0.274 -0.619  0.429
## 2 Stress Regime   -0.0291        -0.353 -0.261 -0.702
## 3 Intermediate Regime  0.0446        -0.589  0.458  0.231

set.seed(123)

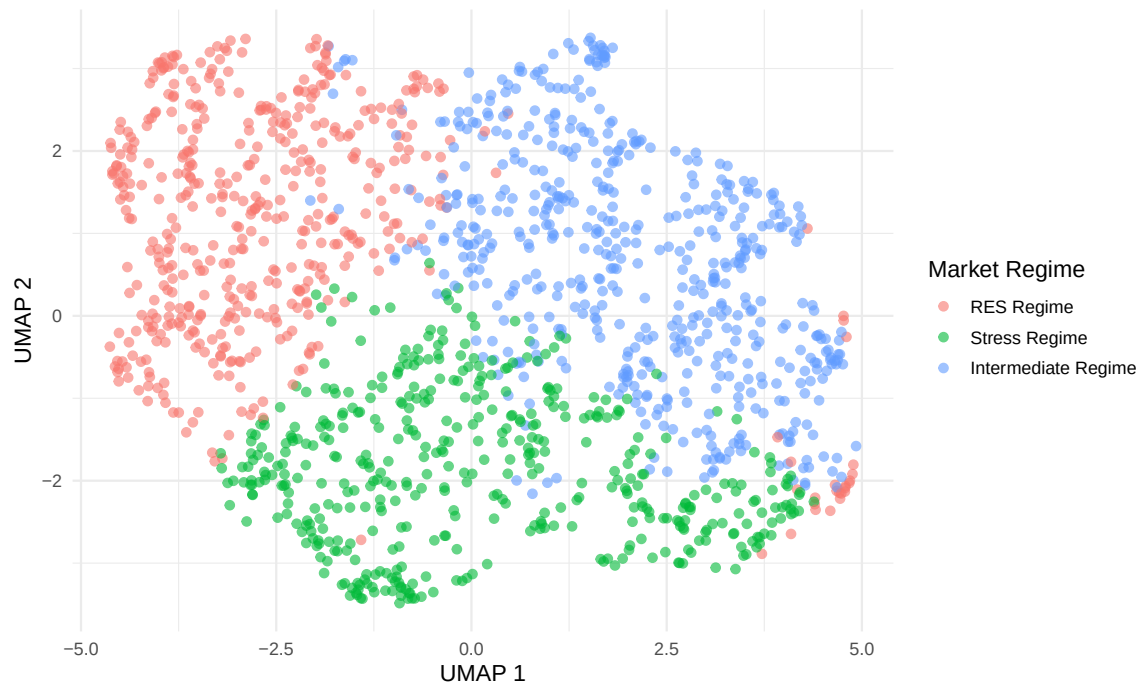
umap_res <- umap(
  X_scaled,
  n_neighbors = 25,
  min_dist = 0.15,
  metric = "euclidean"
)

df_umap <- df_regime %>%
  mutate(
    UMAP1 = umap_res[,1],
    UMAP2 = umap_res[,2]
  )

ggplot(df_umap, aes(UMAP1, UMAP2, color = Cluster)) +
  geom_point(alpha = 0.6, size = 2) +
  theme_minimal(base_size = 13) +
  labs(
    title = "Latent regimes of the European electricity market",
    subtitle = "UMAP projection based on RES, load, fossil energy and flows",
    x = "UMAP 1",
    y = "UMAP 2",
    color = "Market Regime"
  )

```

Latent regimes of the European electricity market
UMAP projection based on RES, load, fossil energy and flows



The UMAP projection reveals the existence of three well-distinct latent regimes of the European electricity market, corresponding to situations dominated by renewable energies, system stress phases, and intermediate configurations, confirming a non-linear structuring of price dynamics.

```
df_umap <- df_umap %>%
  mutate(
    market_regime = factor(
      Cluster,
      levels = sort(unique(Cluster)),
      labels = c(
        "RES Regime (surplus)",
        "Intermediate regime",
        "Stress regime"
      )
    )
  )

df_quant_lines <- df_umap %>%
  group_by(COUNTRY, market_regime) %>%
  summarise(
    q05 = quantile(TARGET, 0.05, na.rm = TRUE),
    q95 = quantile(TARGET, 0.95, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  pivot_longer(
    cols = c(q05, q95),
    names_to = "quantile",
    values_to = "price"
  )
```

```

ggplot(
  df_umap,
  aes(x = TARGET, fill = COUNTRY)
) +

  geom_density(alpha = 0.45, linewidth = 1) +

  geom_vline(
    data = df_quant_lines,
    aes(xintercept = price),
    linetype = "dashed",
    linewidth = 0.7,
    color = "black"
  ) +

  facet_grid(COUNTRY ~ market_regime) +

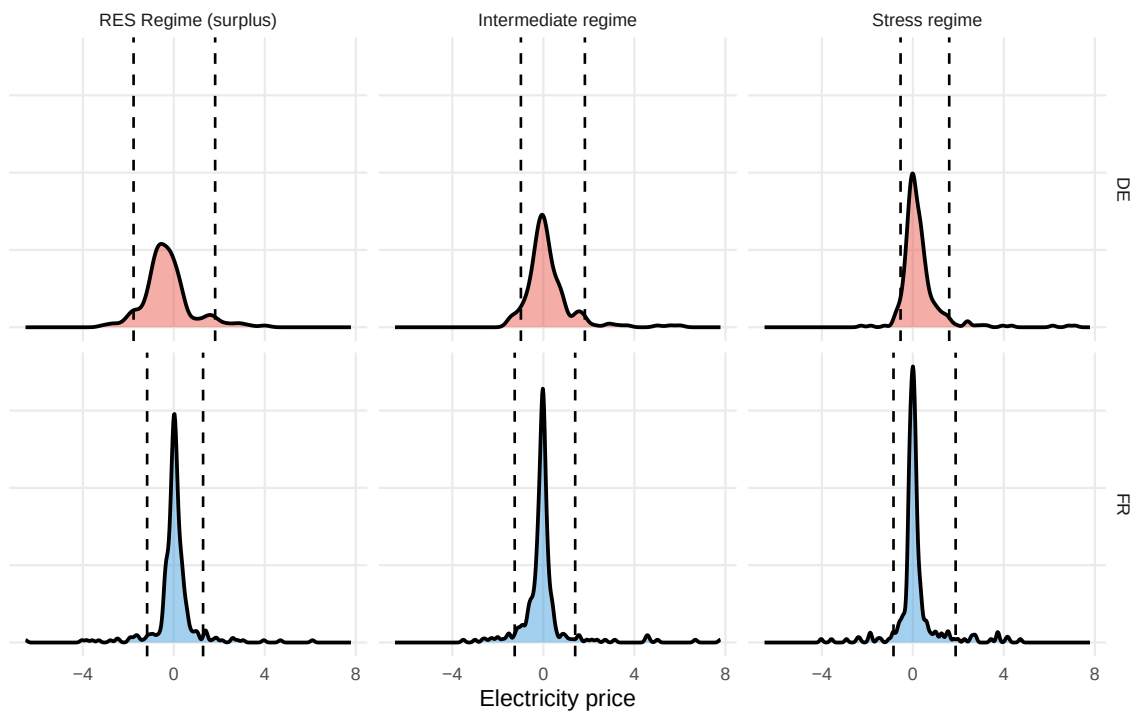
  scale_fill_manual(
    values = c("FR" = "#3498db", "DE" = "#e74c3c"),
    guide = "none"
  ) +

  theme_minimal(base_size = 13) +
  theme(
    axis.text.y = element_blank(),
    axis.title.y = element_blank(),
    panel.grid.minor = element_blank()
  ) +

  labs(
    title = "Price distribution by market regime",
    x = "Electricity price"
  )

```

Price distribution by market regime



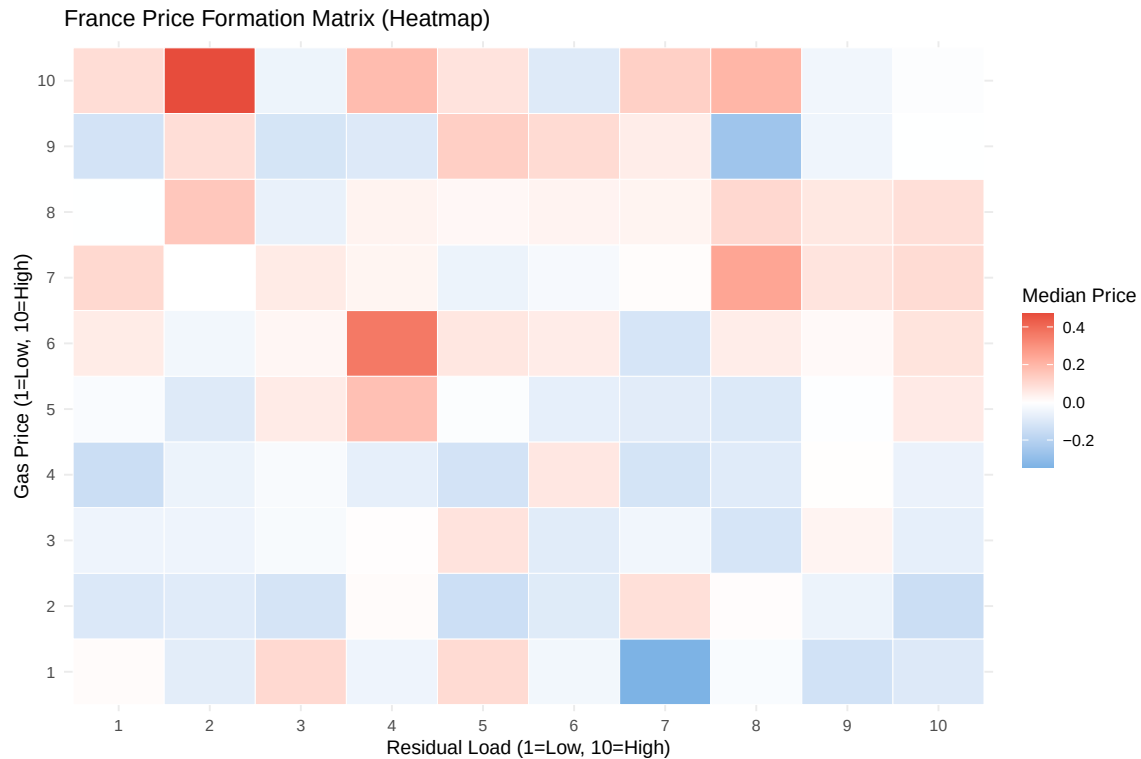
Following the identification of latent regimes by UMAP, the analysis of price distributions shows that each regime is associated with a distinct risk profile, with prices more compressed in the RES regime, moderate dispersion in the intermediate regime, and significantly increased volatility in the stress regime.

```
data_heatmap <- df_train %>%
  filter(COUNTRY == "FR") %>%
  mutate(
    # Create 10 bins for Residual Load and Gas
    Bin_Residual = ntile(FR_RESIDUAL_LOAD, 10),
    Bin_Gas = ntile(GAS_RET, 10)
  ) %>%
  group_by(Bin_Residual, Bin_Gas) %>%
  summarise(Median_Price = median(TARGET), .groups = "drop")

ggplot(data_heatmap, aes(x = factor(Bin_Residual), y = factor(Bin_Gas), fill = Median_Price)) +
  geom_tile(color = "white") + # The tiles

  # Colours: Blue (Low price) -> Red (High price)
  scale_fill_gradient2(low = "#3498db", mid = "white", high = "#e74c3c", midpoint = 0) +

  theme_minimal() +
  labs(
    title = "France Price Formation Matrix (Heatmap)",
    x = "Residual Load (1=Low, 10=High)",
    y = "Gas Price (1=Low, 10=High)",
    fill = "Median Price"
  )
)
```

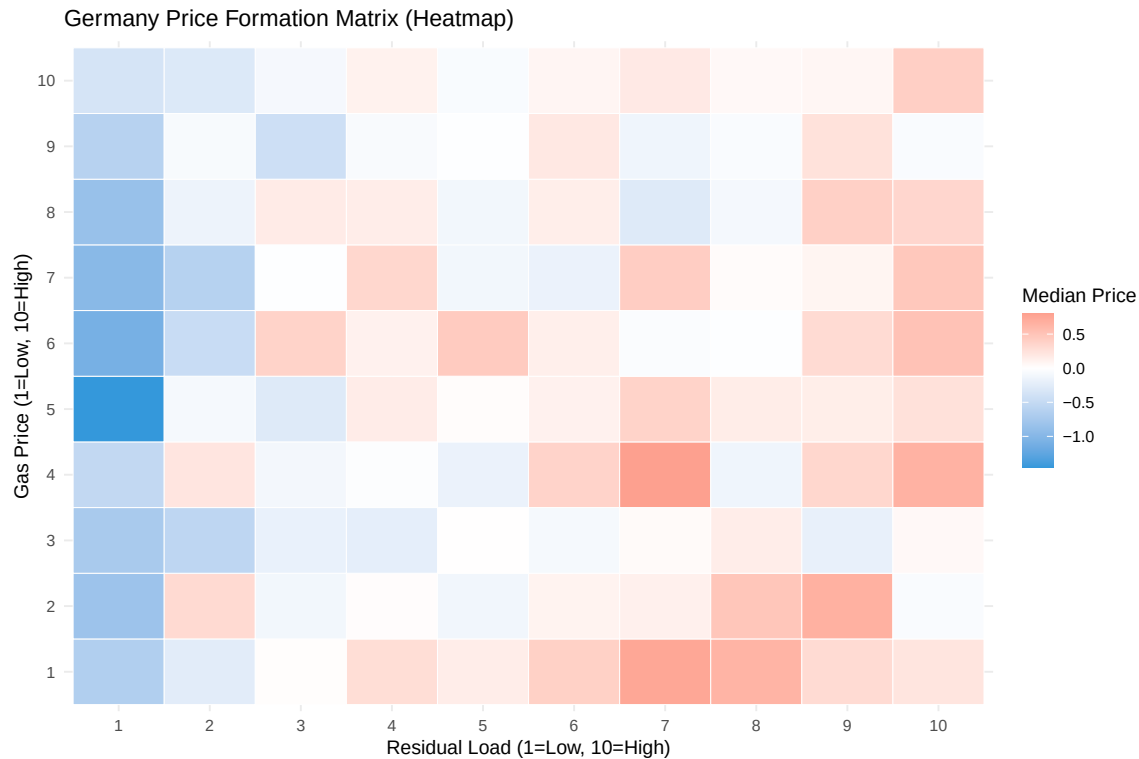


```
data_heatmap <- df_train %>%
  filter(COUNTRY == "DE") %>%
  mutate(
    # Create 10 bins for Residual Load and Gas
    Bin_Residual = ntile(DE_RESIDUAL_LOAD, 10),
    Bin_Gas = ntile(GAS_RET, 10)
  ) %>%
  group_by(Bin_Residual, Bin_Gas) %>%
  summarise(Median_Price = median(TARGET), .groups = "drop")

ggplot(data_heatmap, aes(x = factor(Bin_Residual), y = factor(Bin_Gas), fill = Median_Price)) +
  geom_tile(color = "white") + # The tiles

  # Colours: Blue (Low price) -> Red (High price)
  scale_fill_gradient2(low = "#3498db", mid = "white", high = "#e74c3c", midpoint = 0) +

  theme_minimal() +
  labs(
    title = "Germany Price Formation Matrix (Heatmap)",
    x = "Residual Load (1=Low, 10=High)",
    y = "Gas Price (1=Low, 10=High)",
    fill = "Median Price"
  )
)
```



These price formation matrices show that, in both France and Germany, prices rise sharply when residual load is high and gas is expensive, with a more pronounced and systematic effect in Germany, reflecting increased dependence on fossil fuels under stress conditions.

```
df_fr <- df_train %>% filter(COUNTRY == "FR")

high_thresh <- quantile(df_fr$TARGET, 0.85, na.rm = TRUE)
low_thresh <- quantile(df_fr$TARGET, 0.15, na.rm = TRUE)

features_radar <- c("FR_CONSUMPTION", "FR_WINDPOW", "FR_SOLAR",
                    "FR_NUCLEAR", "FR_GAS", "FR_HYDRO")

labels_radar <- c("Consumption", "Wind", "Solar", "Nuclear", "Gas", "Hydro")

df_norm <- df_fr %>% select(all_of(features_radar))
df_norm <- as.data.frame(lapply(df_norm, function(x)
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
))

mean_high <- colMeans(df_norm[df_fr$TARGET > high_thresh, ], na.rm = TRUE)
mean_low <- colMeans(df_norm[df_fr$TARGET < low_thresh, ], na.rm = TRUE)

radar_data <- data.frame(
  rbind(
    max = rep(1, 6),
    min = rep(0, 6),
    high = mean_high,
    low = mean_low
  )
)
```

```

colnames(radar_data) <- labels_radar

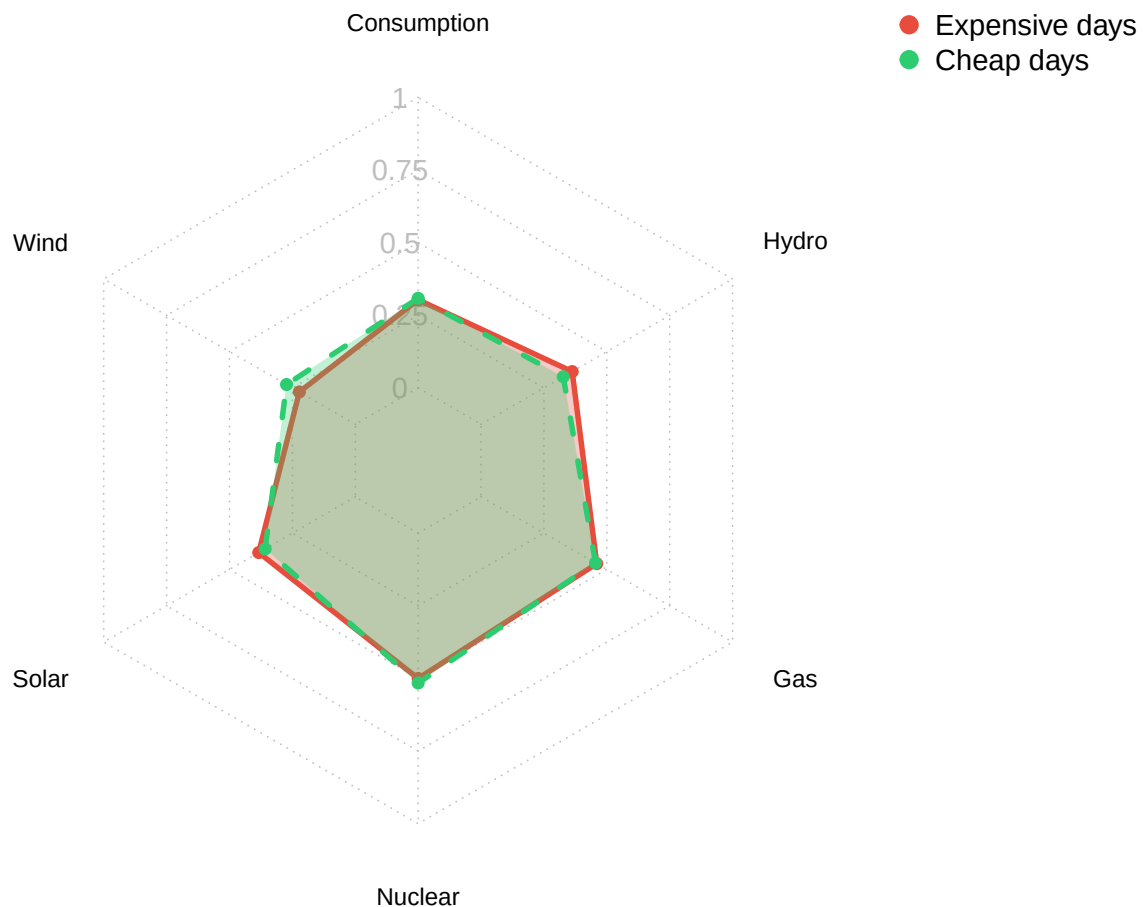
par(mar = c(1, 2, 2, 1))

radarchart(
  radar_data,
  axistype = 1,
  pcol = c("#E74C3C", "#2ECC71"),
  pfc col = c(scales::alpha("#E74C3C", 0.3),
              scales::alpha("#2ECC71", 0.3)),
  plwd = 3,
  cglcol = "grey",
  axislabcol = "grey",
  caxislabels = seq(0, 1, 0.25),
  vlce x = 0.8,
  title = "FRANCE - Profile of expensive vs cheap days"
)

legend(
  "topright",
  legend = c("Expensive days", "Cheap days"),
  col = c("#E74C3C", "#2ECC71"),
  pch = 20,
  pt.cex = 2,
  bty = "n"
)

```

FRANCE – Profile of expensive vs cheap days



```
df_de <- df_train %>% filter(COUNTRY == "DE")

high_thresh <- quantile(df_de$TARGET, 0.85, na.rm = TRUE)
low_thresh <- quantile(df_de$TARGET, 0.15, na.rm = TRUE)

features_radar <- c("DE_CONSUMPTION", "DE_WINDPOW", "DE_SOLAR",
                    "DE_NUCLEAR", "DE_GAS", "DE_HYDRO")

labels_radar <- c("Consumption", "Wind", "Solar", "Nuclear", "Gas", "Hydro")

df_norm <- df_de %>% select(all_of(features_radar))
df_norm <- as.data.frame(lapply(df_norm, function(x)
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
))

mean_high <- colMeans(df_norm[df_de$TARGET > high_thresh, ], na.rm = TRUE)
mean_low <- colMeans(df_norm[df_de$TARGET < low_thresh, ], na.rm = TRUE)

radar_data <- data.frame(
  rbind(
    max = rep(1, 6),
    min = rep(0, 6),
```



```

    high = mean_high,
    low  = mean_low
  )
)

colnames(radar_data) <- labels_radar

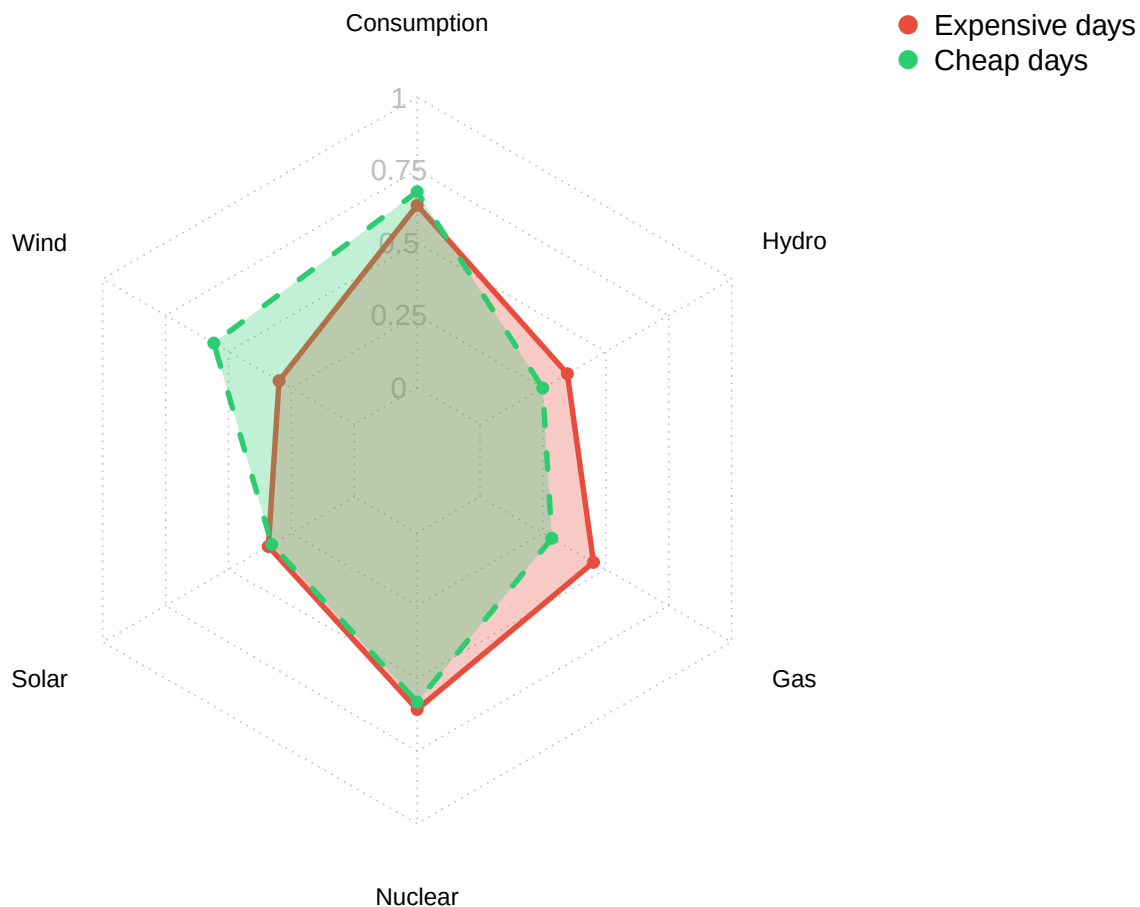
par(mar = c(1, 2, 2, 1))

radarchart(
  radar_data,
  axistype = 1,
  pcol = c("#E74C3C", "#2ECC71"),
  pfccl = c(scales::alpha("#E74C3C", 0.3),
            scales::alpha("#2ECC71", 0.3)),
  plwd = 3,
  cglcol = "grey",
  axislabcol = "grey",
  caxislabels = seq(0, 1, 0.25),
  vlce = 0.8,
  title = "Germany - Profile of expensive vs cheap days"
)

legend(
  "topright",
  legend = c("Expensive days", "Cheap days"),
  col = c("#E74C3C", "#2ECC71"),
  pch = 20,
  pt.cex = 2,
  bty = "n"
)

```

Germany – Profile of expensive vs cheap days



After identifying extreme price configurations via the load–gas matrices, this analysis of average profiles confirms that expensive days are associated with high demand, increased reliance on fossil fuels, and lower contribution from renewables, particularly in Germany.

```
cols_pays <- c("FR" = "#3498db", "DE" = "#e74c3c")

# Data preparation
df_merit <- df_train %>%
  filter(COUNTRY %in% c("FR", "DE")) %>%
  mutate(RENEWABLE = ifelse(COUNTRY == "FR", FR_WINDPOW + FR_SOLAR, DE_WINDPOW + DE_SOLAR)) %>%
  group_by(COUNTRY) %>%
  mutate(RENEW_Q = ntile(RENEWABLE, 4)) %>%
  ungroup()

# Summary calculation (Median + Interval)
df_summary2 <- df_merit %>%
  group_by(COUNTRY, RENEW_Q) %>%
  summarise(
    median_price = median(TARGET),
    q25 = quantile(TARGET, 0.25),
    q75 = quantile(TARGET, 0.75),
    .groups = "drop"
  )
```

```

ggplot(df_summary2, aes(x = factor(RENEW_Q), y = median_price, color = COUNTRY, fill = COUNTRY, group =

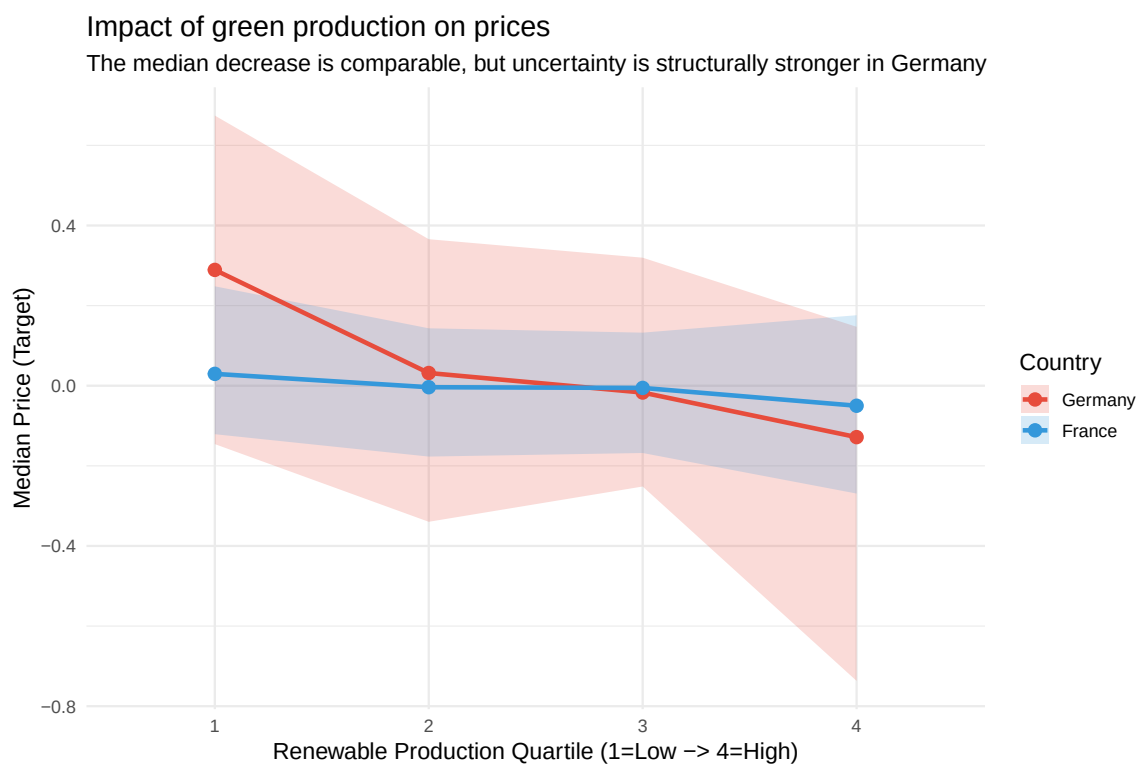
# Uncertainty zone (Ribbon)
geom_ribbon(aes(ymin = q25, ymax = q75), alpha = 0.2, color = NA) +

# Line and Points
geom_line(linewidth = 1.2) +
geom_point(size = 3) +

# Colours and Labels
scale_color_manual(values = cols_pays, labels = c("FR" = "France", "DE" = "Germany")) +
scale_fill_manual(values = cols_pays, labels = c("FR" = "France", "DE" = "Germany")) +

theme_minimal(base_size = 13) +
labs(
  title = "Impact of green production on prices",
  subtitle = "The median decrease is comparable, but uncertainty is structurally stronger in Germany",
  x = "Renewable Production Quartile (1=Low -> 4=High)",
  y = "Median Price (Target)",
  color = "Country",
  fill = "Country"
)

```



After characterising the profiles of expensive and cheap days, the effect of renewable production confirms a downward impact on prices, comparable in median between France and Germany, but structurally more unstable in Germany. This was expected — Germany has a greater weight in renewable energies while nuclear energy in France plays a dampening role in price variation.

```

# Using French wind for France, and German wind for Germany
df_fr_clean <- df_train %>%

```

```

filter(COUNTRY == "FR") %>%
select(TARGET, WIND = FR_WINDPOW) %>%
mutate(Country = "France")

df_de_clean <- df_train %>%
filter(COUNTRY == "DE") %>%
select(TARGET, WIND = DE_WINDPOW) %>%
mutate(Country = "Germany")

# Merge
df_compare <- rbind(df_fr_clean, df_de_clean)

# 3. Creating bins (Deciles)
# Wind split into 10 segments (Deciles) per country
# Segment 1 = The 10% of days with the least wind
# Segment 10 = The 10% of days with the most wind
df_summary <- df_compare %>%
group_by(Country) %>%
mutate(Wind_Bin = ntile(WIND, 10)) %>% # Split into 10 equal groups
group_by(Country, Wind_Bin) %>%
summarise(
  Avg_Price = mean(TARGET),
  Price_SE = sd(TARGET)/sqrt(n()) # For the confidence interval
)

# 4. The Sensitivity Chart
ggplot(df_summary, aes(x = Wind_Bin, y = Avg_Price, color = Country, group = Country)) +

  geom_line(size = 2) +

  geom_point(size = 4) +

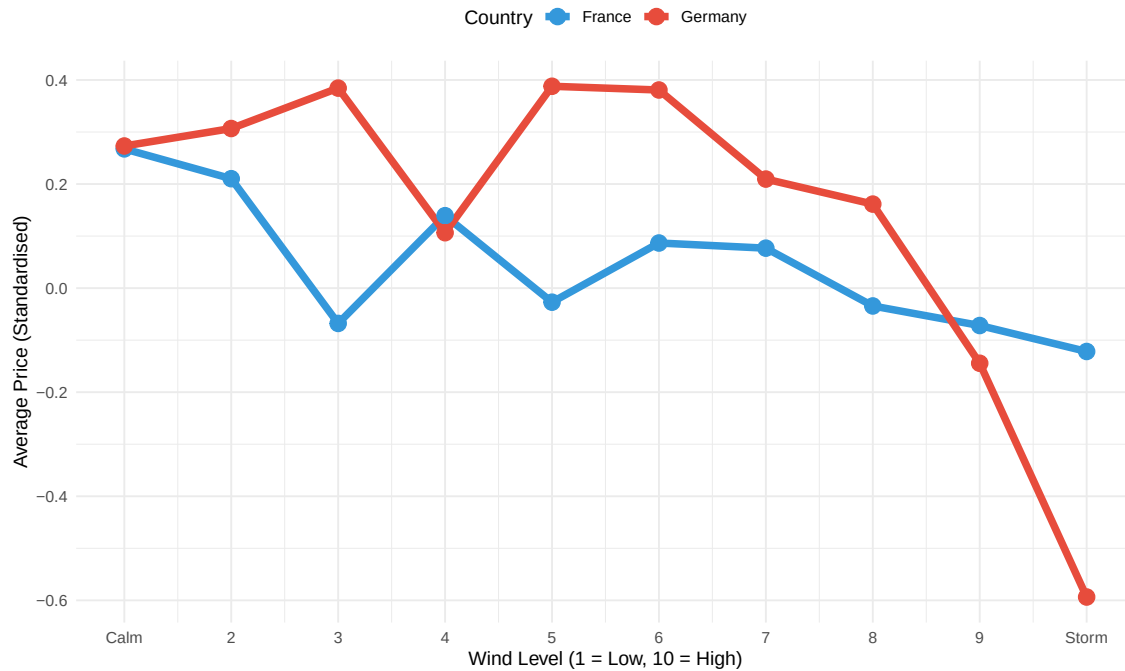
  scale_color_manual(values = c("Germany" = "#E74C3C", "France" = "#3498DB")) +
  scale_x_continuous(breaks = 1:10, labels = c("Calm", "2", "3", "4", "5", "6", "7", "8", "9", "Storm"))

  theme_minimal() +
  labs(
    title = "Who is most dependent on wind?",
    subtitle = "Comparison of price drops as wind increases (Deciles)",
    x = "Wind Level (1 = Low, 10 = High)",
    y = "Average Price (Standardised)"
  ) +
  theme(
    plot.title = element_text(face = "bold", size = 16),
    legend.position = "top"
  )

```

Who is most dependent on wind?

Comparison of price drops as wind increases (Deciles)



Following the analysis of the impact of renewable energies, this comparison shows that increasing wind leads to a much more pronounced and monotonic price drop in Germany than in France, confirming a structurally stronger dependence of the German market on wind power.

```
# Create a unified table
df_fr_clean <- df_train %>%
  filter(COUNTRY == "FR") %>%
  select(TARGET, SOLAR = FR_SOLAR) %>%
  mutate(Country = "France")

df_de_clean <- df_train %>%
  filter(COUNTRY == "DE") %>%
  select(TARGET, SOLAR = DE_SOLAR) %>%
  mutate(Country = "Germany")

# Merge
df_compare <- bind_rows(df_fr_clean, df_de_clean)

# Create solar deciles (by country)
df_summary <- df_compare %>%
  group_by(Country) %>%
  mutate(Solar_Bin = ntile(SOLAR, 10)) %>%
  group_by(Country, Solar_Bin) %>%
  summarise(
    Avg_Price = mean(TARGET),
    Price_SE = sd(TARGET) / sqrt(n()),
    .groups = "drop"
  )

# Solar sensitivity chart
```

```

ggplot(df_summary, aes(x = Solar_Bin, y = Avg_Price, color = Country, group = Country)) +
  geom_line(linewidth = 2) +
  geom_point(size = 4) +

  scale_color_manual(
    values = c("Germany" = "#E74C3C", "France" = "#3498DB")
  ) +

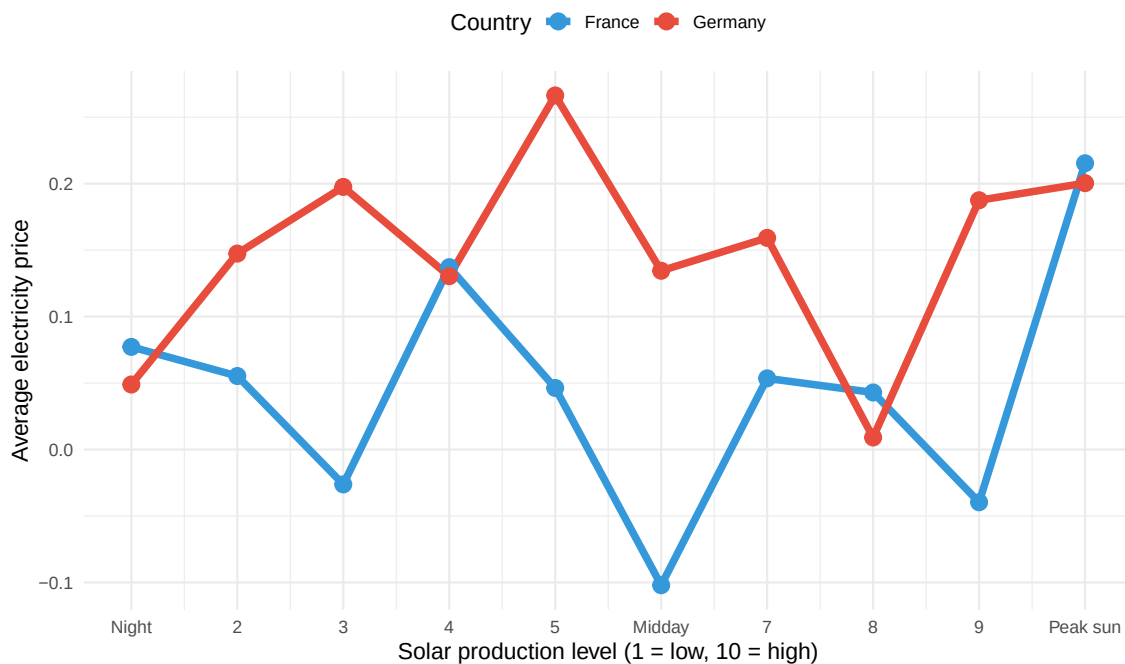
  scale_x_continuous(
    breaks = 1:10,
    labels = c("Night", "2", "3", "4", "5", "Midday", "7", "8", "9", "Peak sun")
  ) +

  theme_minimal(base_size = 13) +
  labs(
    title = "Who is most dependent on solar?",
    subtitle = "Comparison of price variation as solar production increases (Deciles)",
    x = "Solar production level (1 = low, 10 = high)",
    y = "Average electricity price",
    color = "Country"
  ) +
  theme(
    plot.title = element_text(face = "bold", size = 16),
    legend.position = "top"
  )

```

Who is most dependent on solar?

Comparison of price variation as solar production increases (Deciles)



Following on from the wind analysis, this chart shows that the effect of solar on prices is weaker and less monotonic than that of wind, with greater sensitivity in Germany than in France, but without a comparable structural role to that of wind.

```

# Creating the renewable surplus indicator
df_neg <- df_train %>%
  mutate(
    renew_ratio = ifelse(
      COUNTRY == "FR",
      (FR_WINDPOW + FR_SOLAR) / FR_CONSUMPTION,
      (DE_WINDPOW + DE_SOLAR) / DE_CONSUMPTION
    ),
    price_negative = TARGET < 0
  )

# Winsorisation (1% - 99%)
q_low <- quantile(df_neg$renew_ratio, 0.01, na.rm = TRUE)
q_high <- quantile(df_neg$renew_ratio, 0.99, na.rm = TRUE)

df_neg <- df_neg %>%
  mutate(
    renew_ratio_w = pmin(pmax(renew_ratio, q_low), q_high)
  )

# Discretisation into regimes (tertiles)
df_neg <- df_neg %>%
  group_by(COUNTRY) %>%
  mutate(
    renew_regime = ntile(renew_ratio_w, 3),
    renew_regime = factor(
      renew_regime,
      levels = c(1, 2, 3),
      labels = c(
        "Low RES surplus",
        "Moderate RES surplus",
        "High RES surplus"
      )
    )
  ) %>%
  ungroup()

df_neg_summary <- df_neg %>%
  group_by(COUNTRY, renew_regime) %>%
  summarise(
    prob_negative = mean(price_negative),
    n_obs = n(),
    .groups = "drop"
  )

ggplot(df_neg_summary,
  aes(x = renew_regime, y = prob_negative, color = COUNTRY, group = COUNTRY)) +
  geom_line(linewidth = 1.2) +
  geom_point(size = 3) +

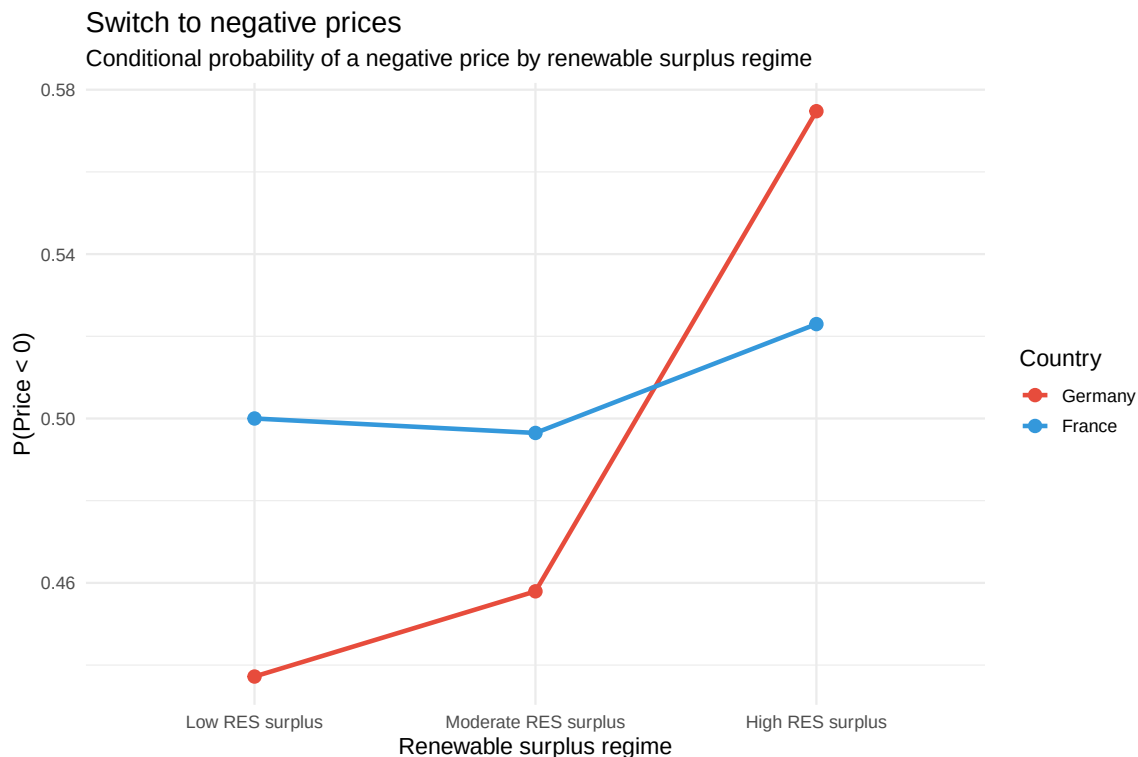
```

```

scale_color_manual(
  values = c("FR" = "#3498db", "DE" = "#e74c3c"),
  labels = c("FR" = "France", "DE" = "Germany")
) +

theme_minimal(base_size = 13) +
labs(
  title = "Switch to negative prices",
  subtitle = "Conditional probability of a negative price by renewable surplus regime",
  x = "Renewable surplus regime",
  y = "P(Price < 0)",
  color = "Country"
)

```



As an extension of the previous analyses, this chart shows that increasing renewable surplus significantly raises the probability of negative prices, an effect particularly pronounced in Germany, confirming a structurally higher sensitivity to RES surpluses.

Feature Engineering

Raw data is not sufficient to capture the non-linearity of electricity prices (e.g., prices explode exponentially and non-linearly during cold spells). We therefore enriched the dataset with the following variables:

1. **Non-Linear Transformations:** Addition of the square of temperatures (TEMP^2) to capture the non-linear effect of heating in winter and air conditioning in summer.
2. **Physical Ratios:** Calculation of the share of renewable energies (RENEW_RATIO) which influences the probability of negative prices.
3. **Economic Interactions:** Creation of interaction variables (e.g., THERMAL_STRESS) modelling the

marginal cost of production. According to the *Merit Order* theory, the price is set by the last plant called upon (often gas or coal).

4. **Temporal Dynamics:** Integration of lag variables ($t - 1$, $t - 2$, $t - 24$) and 24h rolling statistics to capture price inertia and volatility.

```
df_train <- df_train %>%
  group_by(COUNTRY) %>%
  arrange(DAY_ID) %>%
  mutate(
    target_lag1 = lag(TARGET, 1),
    target_lag2 = lag(TARGET, 2),
    target_lag24 = lag(TARGET, 24),
    roll_mean_24 = zoo::rollmean(TARGET, 24, fill = NA, align = "right"),
    roll_sd_24 = zoo::rollapply(TARGET, 24, sd, fill = NA, align = "right")
  ) %>%
  ungroup()

# Feature selection and creation functions for France and Germany
select_features_de <- function(df) {
  df %>% select(TARGET, starts_with("DE_"), GAS_RET, COAL_RET, CARBON_RET,
               target_lag1, target_lag2, target_lag24,
               roll_mean_24, roll_sd_24) %>%
  select(-DE_WIND, -DE_RAIN, -DE_NET_EXPORT, -DE_FR_EXCHANGE)
}

select_features_fr <- function(df) {
  df %>% select(TARGET, starts_with("FR_"), GAS_RET, COAL_RET, CARBON_RET,
               DE_WINDPOW, DE_SOLAR, DE_CONSUMPTION, DE_NET_IMPORT, DE_RESIDUAL_LOAD,
               target_lag1, target_lag2, target_lag24,
               roll_mean_24, roll_sd_24) %>%
  select(-FR_WIND, -FR_RAIN, -FR_NET_EXPORT, -FR_DE_EXCHANGE)
}

# Creation of non-linear variables
add_nonlinear_features <- function(df, country) {
  epsilon <- 1e-5
  if (country == "FR") {
    df <- df %>% mutate(
      FR_TEMP_sq = FR_TEMP^2,
      FR_THERMAL_STRESS = FR_RESIDUAL_LOAD * GAS_RET,
      FR_RENEW_RATIO = (FR_WINDPOW + FR_SOLAR) / (FR_CONSUMPTION + epsilon),
      FR_HEATING_NEED = pmax(0, 15 - FR_TEMP)
    )
  } else {
    df <- df %>% mutate(
      DE_TEMP_sq = DE_TEMP^2,
      DE_THERMAL_STRESS = DE_RESIDUAL_LOAD * COAL_RET,
      DE_RENEW_RATIO = (DE_WINDPOW + DE_SOLAR) / (DE_CONSUMPTION + epsilon)
    )
  }
  return(df)
}

# Pipeline application for France and Germany
train_fr_final <- df_train %>%
```

```

filter(COUNTRY == "FR") %>%
select_features_fr() %>%
add_nonlinear_features("FR")

train_de_final <- df_train %>%
  filter(COUNTRY == "DE") %>%
  select_features_de() %>%
  add_nonlinear_features("DE")

```

Feature Selection

This code defines a cleaning function that optimises the French and German datasets by removing near-zero variance variables and those showing strong multicollinearity. This procedure reduces dimensionality and improves the stability of future training models by retaining only the most relevant predictors.

```

# Generic cleaning function before training
clean_features_pre_training <- function(df, target_col = "TARGET", corr_thresh = 0.8) {

  cat(paste("--- Cleaning:", deparse(substitute(df)), "---\n"))
  cat("Initial number of variables:", ncol(df), "\n")

  # 1. Separate Target and Features
  y <- df[[target_col]]
  X <- df %>% select(-all_of(target_col))

  # 2. Remove near-zero variance variables (those that never change)
  nzv <- nearZeroVar(X)
  if (length(nzv) > 0) {
    cat("Near-zero variance variables removed:", names(X)[nzv], "\n")
    X <- X[, -nzv]
  }

  # 3. Remove Multicollinearity (Redundancy between features)
  # Compute the correlation matrix between explanatory variables only
  corr_mat <- cor(X, use = "pairwise.complete.obs", method = "spearman")

  # findCorrelation returns the indices of columns to remove
  high_corr_idx <- findCorrelation(corr_mat, cutoff = corr_thresh)

  if (length(high_corr_idx) > 0) {
    removed_vars <- names(X)[high_corr_idx]
    cat("Highly correlated variables removed (>", corr_thresh, "):\n")
    print(removed_vars)
    X <- X[, -high_corr_idx]
  }

  # 4. Reconstruct the clean DataFrame
  df_clean <- bind_cols(X, TARGET = y)

  cat("Final number of variables:", ncol(df_clean), "\n\n")
  return(df_clean)
}

```

```
# --- APPLICATION TO BOTH COUNTRIES ---
train_fr_selected <- clean_features_pre_training(train_fr_final, corr_thresh = 0.80)

## --- Cleaning: train_fr_final ---
## Initial number of variables: 28
## Highly correlated variables removed (> 0.8 ):
## [1] "FR_CONSUMPTION"    "FR_RESIDUAL_LOAD" "DE_SOLAR"          "FR_HEATING_NEED"
## Final number of variables: 24

train_de_selected <- clean_features_pre_training(train_de_final, corr_thresh = 0.80)

## --- Cleaning: train_de_final ---
## Initial number of variables: 23
## Final number of variables: 23
```

XGBOOST Modelling

This code implements a random search optimisation combined with 5-fold cross-validation to tune the hyperparameters of the XGBoost models for France and Germany. Using a rank transformation of the targets, the algorithm specifically maximises the Spearman correlation, thereby ensuring robust and generalisable performance before retraining the final model on all the data.

```
# =====
# XGBOOST WITH CROSS-VALIDATION (5 FOLDS)
# =====

run_xgb_cv_tuning <- function(df_train, country_name, n_iter = 20, k_folds = 5) {

  cat(paste("\n=== Starting CV Tuning (", k_folds, "folds) for", country_name, "===\n"))

  # 1. Create Folds (Indices)
  set.seed(123)
  folds <- createFolds(df_train$TARGET, k = k_folds, list = TRUE)

  best_avg_score <- -1
  best_params <- list()
  best_fold_scores <- numeric(k_folds)

  # 2. Random Search
  for (i in 1:n_iter) {

    # Parameter lists
    params <- list(
      booster      = "gbtree",
      objective     = "reg:squarederror",
      eta          = runif(1, 0.01, 0.1),
      max_depth    = sample(4:12, 1),
      subsample    = runif(1, 0.6, 1.0),
      colsample_bytree = runif(1, 0.6, 1.0),
      min_child_weight = sample(1:10, 1)
    )

    # Score storage
    current_fold_scores <- numeric(k_folds)
```

```

# 3. Cross-Validation Loop
for (j in 1:k_folds) {
  # Indices for this fold
  val_idx <- folds[[j]]

  # Split
  train_fold <- df_train[-val_idx, ]
  val_fold <- df_train[val_idx, ]

  dtrain <- xgb.DMatrix(data = as.matrix(train_fold %>% select(-TARGET)),
                        label = rank(train_fold$TARGET)) # Ranking Trick for Spearman correlation
  dval <- xgb.DMatrix(data = as.matrix(val_fold %>% select(-TARGET)),
                      label = val_fold$TARGET)

  # Training
  model <- xgb.train(params = params, data = dtrain, nrounds = 500,
                    early_stopping_rounds = 20, watchlist = list(val=dval), verbose = 0)

  # Spearman Score
  preds <- predict(model, dval)
  current_fold_scores[j] <- cor(val_fold$TARGET, preds, method = "spearman")
}

# Average of 5 folds
avg_score <- mean(current_fold_scores)

# If this is the best parameter set, save it
if (avg_score > best_avg_score) {
  best_avg_score <- avg_score
  best_params <- params
  best_fold_scores <- current_fold_scores

  # Compact display for tracking
  cat(paste0("Iter ", i, " [New Best]: Mean=", round(avg_score, 4),
            " | Std=", round(sd(current_fold_scores), 4), "\n"))
}
}

# 4. Final Detailed Display
cat("\n-----\n")
cat(paste("FINAL CV RESULTS -", country_name, "\n"))
cat("Scores by Fold: ", paste(round(best_fold_scores, 4), collapse = " | "), "\n")
cat("Global Mean: ", round(best_avg_score, 4), "\n")
cat("Standard Deviation (Std): ", round(sd(best_fold_scores), 4), "\n")
cat("-----\n")

# 5. Final Retraining (Refit)
# Train a single model on 100% of data with the best parameters
# to allow subsequent factor interpretation.
cat("Retraining the final model on the entire dataset...\n")

ddf_train <- xgb.DMatrix(data = as.matrix(df_train %>% select(-TARGET)),
                        label = rank(df_train$TARGET))

```

```

    final_model <- xgb.train(params = best_params, data = ddf_train, nrounds = 600, verbose = 0)

    return(list(model = final_model, score = best_avg_score, fold_scores = best_fold_scores, std = sd(best_fold_scores)))
  }

# Launch with CV
res_fr <- run_xgb_cv_tuning(train_fr_selected, "France", n_iter = 50, k_folds = 5)

##
## === Starting CV Tuning ( 5 folds) for France ===
## Iter 1 [New Best]: Mean=0.0804 | Std=0.0479
## Iter 2 [New Best]: Mean=0.0997 | Std=0.0664
## Iter 3 [New Best]: Mean=0.1347 | Std=0.0907
## Iter 25 [New Best]: Mean=0.159 | Std=0.0592
##
## -----
## FINAL CV RESULTS - France
## Scores by Fold: 0.1563 | 0.1527 | 0.2083 | 0.0655 | 0.212
## Global Mean: 0.159
## Standard Deviation (Std): 0.0592
## -----
## Retraining the final model on the entire dataset...
res_de <- run_xgb_cv_tuning(train_de_selected, "Germany", n_iter = 50, k_folds = 5)

##
## === Starting CV Tuning ( 5 folds) for Germany ===
## Iter 1 [New Best]: Mean=0.2848 | Std=0.0571
## Iter 5 [New Best]: Mean=0.3575 | Std=0.0926
##
## -----
## FINAL CV RESULTS - Germany
## Scores by Fold: 0.3831 | 0.3211 | 0.3709 | 0.2295 | 0.4829
## Global Mean: 0.3575
## Standard Deviation (Std): 0.0926
## -----
## Retraining the final model on the entire dataset...

# Create results table for the report
results_cv <- data.frame(
  Country = c("France", "Germany"),
  Mean_Spearman = c(res_fr$score, res_de$score),
  Std_Dev = c(res_fr$std, res_de$std),
  Fold_1 = c(res_fr$fold_scores[1], res_de$fold_scores[1]),
  Fold_2 = c(res_fr$fold_scores[2], res_de$fold_scores[2]),
  Fold_3 = c(res_fr$fold_scores[3], res_de$fold_scores[3]),
  Fold_4 = c(res_fr$fold_scores[4], res_de$fold_scores[4]),
  Fold_5 = c(res_fr$fold_scores[5], res_de$fold_scores[5])
)

```

```
kable(results_cv, caption = "Detailed Performance (5-Fold Cross-Validation)", digits = 4) %>%
  kable_styling(full_width = F)
```

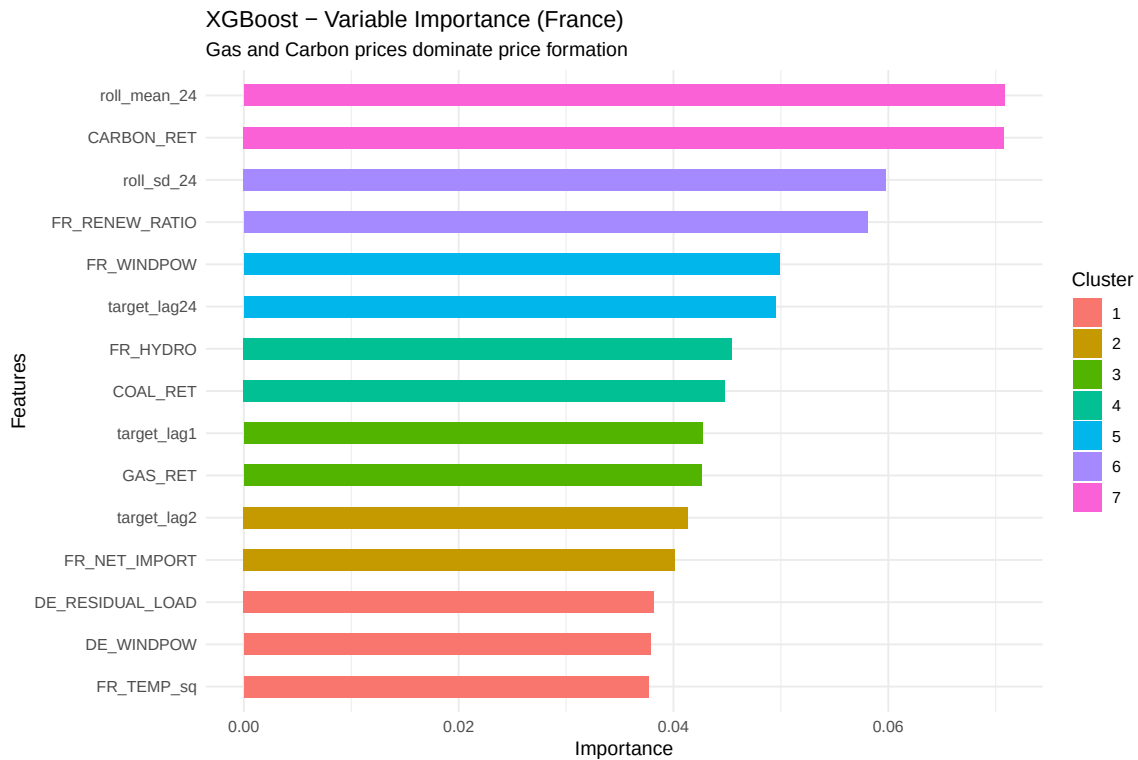
Table 3: Detailed Performance (5-Fold Cross-Validation)

Country	Mean_Spearman	Std_Dev	Fold_1	Fold_2	Fold_3	Fold_4	Fold_5
France	0.1590	0.0592	0.1563	0.1527	0.2083	0.0655	0.2120
Germany	0.3575	0.0926	0.3831	0.3211	0.3709	0.2295	0.4829

XGBoost more effectively captures price dynamics in Germany than in France, consistent with the more unstable and renewable-driven structure of the German market.

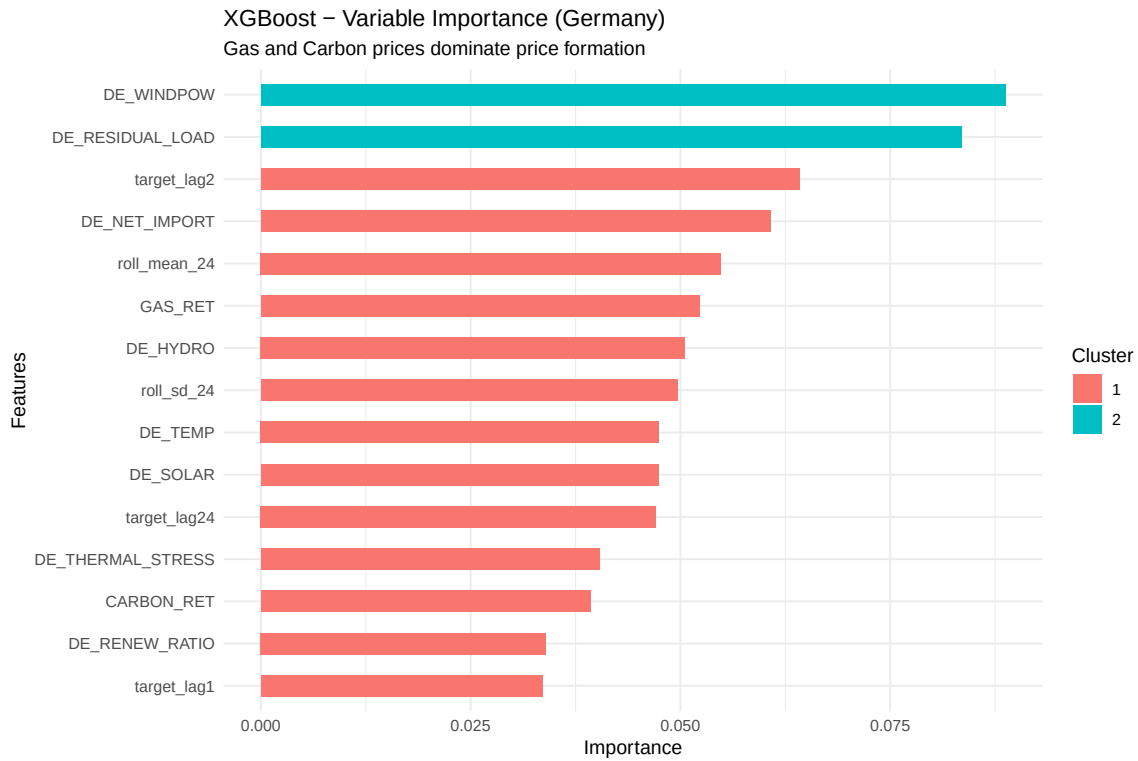
```
# Extracting variable importance for France
importance_matrix_fr <- xgb.importance(model = res_fr$model)

xgb.ggplot.importance(importance_matrix_fr, top_n = 15) +
  theme_minimal() +
  labs(title = "XGBoost - Variable Importance (France)",
        subtitle = "Gas and Carbon prices dominate price formation")
```



```
# Extracting variable importance for Germany
importance_matrix_de <- xgb.importance(model = res_de$model)

xgb.ggplot.importance(importance_matrix_de, top_n = 15) +
  theme_minimal() +
  labs(title = "XGBoost - Variable Importance (Germany)",
        subtitle = "Gas and Carbon prices dominate price formation")
```



XGBoost importances highlight two distinct price formation logics: in France, prices are primarily driven by fuel and carbon costs, while in Germany, physical system constraints — notably wind, residual load, and flows — play a predominant role, confirming a structural dependence on renewable energies.